Truthy vs Falsy Values?

Python's inherent 'truth values': Pythonic gatekeeper & filtering 'Ops

Falsy Values — evaluate to False

?

Truthy Values — evaluate to True

?

Using `bool(x)` converts any object to **True** or **False**

```
def truly():
    _str = input('What: ')
    print(f'Length is {len(_str)}')
    if(_str):
        print("Truthy!")
    else:
        print("Fasly...")
```

```
>>> truly()
What:
Length is 0
Fasly...
>>> truly()
What:
Length is 2
Truthy!
```

```
>>> isinstance(1,object)
True
>>> isinstance('',object)
True
>>> bool(1)
True
>>> bool('')
False
```

Truthy vs Falsy Values?

Python's inherent 'truth values': The key to understanding gatekeeper & filtering 'Ops

Falsy Values — evaluate to False

`None`	the null object
`False`	the boolean False
`0  0.0`	zero in any numeric type
`""`	empty string
`{}[]()`	any empty collections
`range(0)`	empty range (et al.)

Truthy Values — evaluate to True

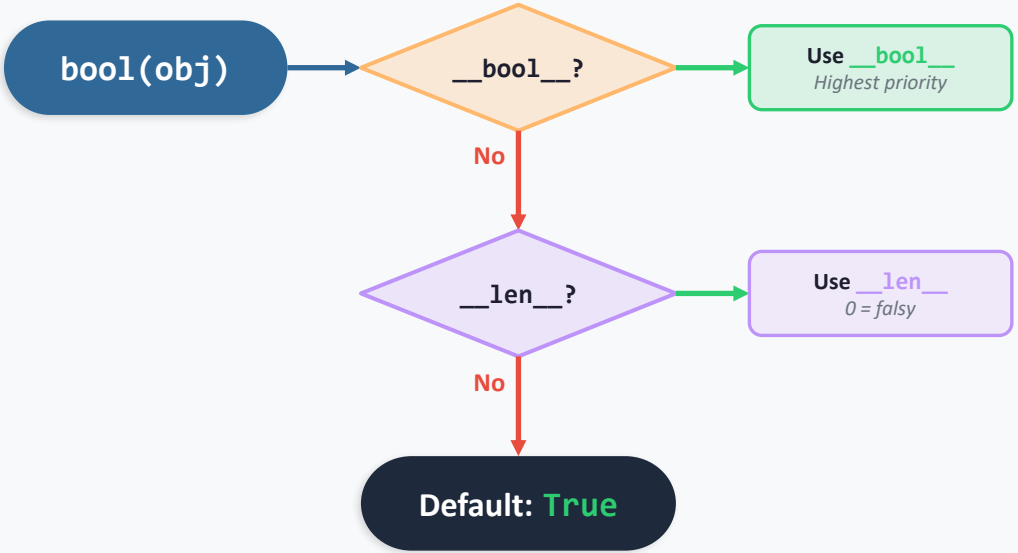
`True`	the boolean True
`1  -3  3.14`	any non-zero number
`"hello"  " "`	any non-empty string
`[1, 2]  [0]`	any non-empty list
`{"a": 1}`	any non-empty dict
`object()`	custom objects (by default)
`lambda: ...`	functions and classes

Everything not in the falsy list is truthy!

Using `bool(x)` converts any object to **True** or **False**

Evaluation Framework

The priority chain Python follows when **bool()** is called on any object



Totally Pythonic Logic!

```
>>> hasattr(1, '__bool__')
...
True
>>> hasattr(1, '__len__')
...
False
>>> hasattr('', '__bool__')
...
False
>>> hasattr('', '__len__')
...
True
```

```
__class__
__delattr__
__dir__
__doc__
__eq__
__format__
__ge__
__getattribute__
__getstate__
__gt__
__hash__
__init__
__init_subclass__
__le__
__lt__
__ne__
__new__
__reduce__
__reduce_ex__
__repr__
__setattr__
__sizeof__
__str__
__subclasshook__
```

D.I.Y Truth: __bool__

Classes can define their own truth value by implementing __bool__ (or __len__)

```
1  class BlankStr:
2      def __init__(self, a_str=None):
3          if not a_str:
4              self.a_str = ''
5          else:
6              self.a_str = str(a_str)
7
8      def __bool__(self):
9          if self.a_str.isnumeric():
10             return bool(int(self.a_str))
11         return not self.a_str.isspace()
```

Fallback: if __bool__ is not defined, Python checks __len__ — an object with length 0 is falsy

The __len__ Fallback

When **__bool__** isn't defined, Python uses **__len__** to determine truth value

```
1  class BlankStr:
2      def __init__(self, a_str=None):
3          if not a_str:
4              self.a_str = ''
5          else:
6              self.a_str = str(a_str)
7
8      def __len__(self):
9          if self.a_str.isspace():
10             return 0
11         return len(self.a_str)
```

No **__bool__**? Python falls back to **__len__** — if length is 0, the object is **falsy**; otherwise **truthy**

Classic “Wrapper Class”

```
1  class BlankStr:
2      def __init__(self, a_str=None):
3          if not a_str:
4              self.a_str = ''
5          else:
6              self.a_str = str(a_str)
7
8      def __bool__(self):
9          if self.a_str.isnumeric():
10             return bool(int(a_str))
11         return not self.a_str.isspace()
12
13     def __len__(self):
14         if self.a_str.isspace():
15             return 0
16         return len(self.a_str)
```

Missions/2026_06_01_TruthyVsF

github.com/TotalPythoneering/Missions/tree/main/2026_06_01_TruthyVsFalsy/src

TotalPythoneering / Missions

Type / to search

Code | Issues | Pull requests | Agents | Actions | Projects | Wiki | Security and quality | Insights | Settings

Files

main

Go to file T

- 2026_06_01_TruthyVsFalsy
  - src
    - blank_str.py
    - input_demo.py
    - input_demo02.py
    - tc01.py
    - tc02.py
  - 2025_05_20_TruthyVsFal...
  - README.md
  - logger.log
- README.md

Missions / 2026_06_01_TruthyVsFalsy / src /

Add file

soft9000 Add files via upload — e900486 · 4 minutes ago — History

Name	Last commit message	Last commit date
..		
blank_str.py	Add files via upload	4 minutes ago
input_demo.py	Add files via upload	4 minutes ago
input_demo02.py	Add files via upload	4 minutes ago
tc01.py	Add files via upload	4 minutes ago
tc02.py	Add files via upload	4 minutes ago

Caveat: “Operator Overloading”

```
 9 class BlankStr:
10     def __init__(self, a_str=None):
11         if not a_str:
12             self.a_str = None
13         else:
14             self.a_str = str(a_str)
15
16     def __eq__(self, obj):
17         raise Exception("Yikes!")
18
```

Thank You!

Go write something Truthy!

Questions? Let's discuss!

```
class BlankStr:
    def __init__(self, a_str=None):
        if not a_str:
            self.a_str = None
        else:
            self.a_str = str(a_str)
##
##    def __bool__(self):
##        if not self.a_str: return False
##        if self.a_str.isnumeric():
##            return bool(int(self.a_str))
##        return not self.a_str.isspace()

    def __len__(self):
        if not self.a_str:
            return 0
        if self.a_str.isspace():
            return 0
```